

Лабораторная работа №6. Продвинутый HTML. Введение в CSS стили.

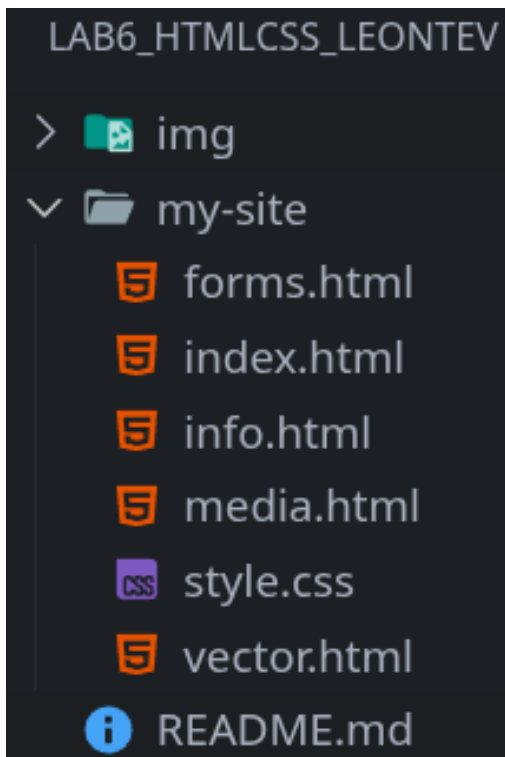
Создание простого сайта про историю ЭВМ

Шаг 1. Подготовка к работе

1. Откройте **Visual Studio Code**.
2. Откройте **терминал**.
3. Перейдите в папку с **документами**.
4. Создайте директорию для лабораторной работы:

Lab6_HtmlCSS_FIO (где *FIO* — ваши фамилия).

5. **Перейдите** в созданную директорию.



6. Создайте **папки**:

- **img**
- **my-site**

И файл:

- **README.md**

Внутри **my-site** создайте следующие файлы:

- **info.html**
- **index.html**
- **media.html**
- **forms.html**
- **vector.html**
- **style.css**

Лайфхак: чтобы создать несколько файлов внутри папки, не переходя в неё, можно воспользоваться следующей командой:

```
➤ <img alt="Terminal icon" data-bbox="93 700 110 720"/>>$ touch my-site/{forms.html,index.html,info.html,media.html,style.css,vector.html}
```

7. Подключите рабочий каталог в VS Code
8. Инициализируйте репозиторий.
9. Добавьте файлы в индекс.
10. Создайте первый коммит.

git commit -m "Initial commit"

11. Подготовьте репозиторий на **GitHub**:

- Создайте новый пустой **public**-репозиторий:

Lab6-HtmlCSS_FIO

➤ Без README, .gitignore и License.

12. Привяжите локальный репозиторий к удалённому и запустите файлы.

13. Сделайте скриншот терминала с выполненными командами **коммита**, **подключения** и **пуша** и сохраните файл в папку **img**:

gitPushLab6_FIO.png

14. Добавьте файлы проекта:

git add .

15. Создайте коммит:

git commit -m "Added push screenshot and project update"

16. Отправьте изменения:

git push

Шаг 2. Создание простой html-страницы

1. Откройте файл **index.html**

2. Создайте пустую страницу. Наберите **!** и нажмите **Enter**

3. Изменяем язык документа

Замените атрибут **lang="en"** на **"ru-RU"**

Шаг 3. Блочные и строчные теги в HTML. Теги **div** и **span**

В HTML все элементы делятся на **блочные** и **строчные**.

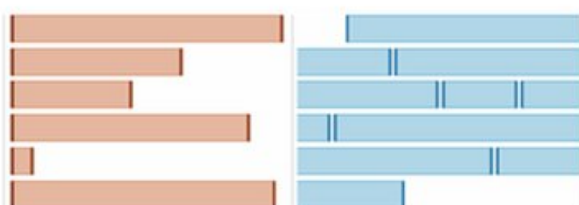


Блочные теги (div, p, h1 и др.):

- занимают всю доступную ширину страницы;
- всегда начинаются с новой строки;
- могут содержать внутри другие теги.

BLOCK

INLINE



Строчные теги (span, a, strong и др.):

- занимают только необходимую ширину;
- не переносят строку;
- используются для выделения части текста.

Теги **div** и **span** не имеют собственного визуального оформления и чаще всего используются для группировки элементов и применения CSS-стилей.

hello

hello

hello

hello

Пример БЕЗ стилей

Пример СО стилями

Рассмотрим простой пример. Напишем в нашем файле **index.html** внутри **body**:

История ЭВМ

Первые электронные вычислительные машины появились в середине XX века.

Тег **span** используется для выделения части текста внутри строки.

```
<body>
  <div>
    <h2>История ЭВМ</h2>
    <p>
      Первые <span>электронные вычислительные машины</span> появились в середине XX века.
    </p>
  </div>

  <div>
    <p style="background-color: green;">
      Тег <span style="background-color: yellow;">span</span> используется для выделения части
текста внутри строки.
    </p>
  </div>
</body>
```

➤ **div** используется как контейнер для логических блоков страницы. Занимает всю ширину, начинается с новой строки;

➤ **span** применяется для выделения отдельных слов или фрагментов текста. занимает только ширину текста, не переносится

Откройте страницу в браузере. Для наглядности я применил изменение цвета, чтобы показать, как меняются теги **p** и **span**:

История ЭВМ

Первые электронные вычислительные машины появились в середине XX века.

Тег **span** используется для выделения части текста внутри строки.

Шаг 4. Теги форматирования текста

HTML предоставляет специальные теги для форматирования текста.

- **** — жирный текст (визуально);
- **** — логическое выделение важного текста;
- **<i>** — курсив (визуально);
- **** — логическое выделение (курсив);

- `<s>` — зачёркнутый текст;
- `<u>` — подчёркнутый текст;
- `<sub>` — нижний индекс;
- `<sup>` — верхний индекс;
- `<small>` — уменьшенный текст;
- `<mark>` — выделение маркером;
- `<br>` — перенос строки;
- `<hr>` — горизонтальная линия;
- `<code>` — отображение кода.

Напишите следующий код, чтобы рассмотреть пример использования тегов форматирования и отформатируйте его по образцу:

ЭВМ — это электронная вычислительная машина.

Важно: первые ЭВМ занимали целые помещения.

Курсивный текст и логически выделенный текст.

Устаревшая информация

Подчёркнутый текст

Формула воды: H₂O

Степень числа: 2³ = 8

Мелкий текст для примечаний

Выделенный важный фрагмент

Первая строка

Вторая строка

Пример кода:

```
<h1>История ЭВМ</h1>
```

ЭВМ — это электронная вычислительная машина.

Важно: первые ЭВМ занимали целые помещения.

Курсивный текст и логически выделенный текст.

Устаревшая информация

Подчёркнутый текст

Формула воды: H₂O

Степень числа: 2³ = 8

Мелкий текст для примечаний

Выделенный важный фрагмент

Первая строка

Вторая строка

Пример кода:

```
<h1>История ЭВМ</h1>
```

Для последней строки, нужно ввести следующую конструкцию:

```
<code>
|   &lt;h1&gt;История ЭВМ&lt;/h1&gt;
| </code>
```

Выполните в терминале:

git add .

git commit -m "Added block, inline elements and formatting tags"

git push

Шаг 5. Абсолютные и относительные ссылки в HTML. Якорные ссылки, mailto, tel, download

В HTML ссылки создаются с помощью тега `<a>`.

1. Относительные ссылки — ведут на файлы внутри проекта (например, `info.html`, `media.html`);

2. Абсолютные ссылки — ведут на внешние ресурсы (например, сайты в интернете);

3. Якорные ссылки — позволяют перейти к определённому месту страницы;

➤ **mailto:** — открывает почтовый клиент для отправки письма;

➤ **tel:** — позволяет позвонить с мобильного устройства;

➤ **download** — используется для скачивания файлов.

Пример кнопки-ссылки на страницу проекта:

```
10 <body>
11 > <div> ... скрытый код
40 </div>
41 <a href="info.html">
42 <button>Перейти к истории ЭВМ</button>
43 </a>
44 </body>
```

Пример кнопки возврата назад (на другой странице):

Пишем в файле **info.html**:

```
index.html М info.html М X
my-site > info.html > a
You, 1 second ago | 1 author (You)
1 <a href="index.html"> You, 3 seconds
2 <button>Вернуться на главную</button>
3 </a>
```

Пример якорной ссылки:

В самом начале файла **index.html** после тега **body** добавим якорь:

```
10 <body>
11 <a href="#generation">Поколения ЭВМ</a>
12 > <div> ...
41 </div>
```

В низу перед закрывающимся тегом **body** добавим:

```
46 <hr>
47 <h2 id="generation">Поколения ЭВМ</h2>
48 <p>Описание поколений электронных вычислительных машин.</p>
49 </body>
```

Чтобы увидеть работу в браузере - увеличьте масштаб (**Ctrl+колёсико**)

Важно:

- **id** должен быть уникальным (не повторяться на странице)
- Якорные ссылки работают без перезагрузки страницы

Пример ссылки на скачивание файла:

```
<a href="..README.md" download>
  Скачать описание лабораторной
</a>
```

Пример ссылки на электронную почту:

```
<a href="mailto:denis.leontev92@yandex.ru">
  Написать письмо
</a>
```

Пример ссылки на телефон:

```
<a href="tel:+88005553535">
  Позвонить
</a>
```

Шаг 6. HTML изображения. Относительные и абсолютные пути

Изображения в HTML добавляются с помощью тега ****.

- **Относительный путь** — путь к файлу внутри проекта;
- **Абсолютный путь** — полный URL к изображению в интернете.

Изображения рекомендуется хранить в отдельной папке (**img**).

Практическое задание

Найдите в интернете изображения поколений ЭВМ (можно на Яндекс.Картинках или Google Images).

Требования:

- **Количество:** 4 изображения (I, II, III, IV поколения)
- **Формат:** JPG или PNG
- **Размер:** примерно 300-500px по ширине

➤ Названия файлов:

computer_generation_1.jpg — первое поколение

computer_generation_2.jpg — второе поколение

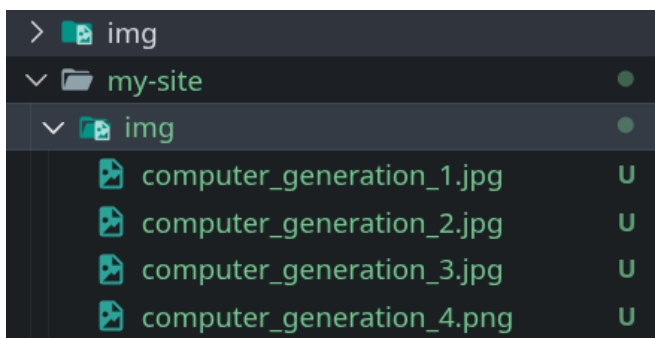
computer_generation_3.jpg — третье поколение

computer_generation_4.jpg — четвертое поколение

Как скачать:

1. Найдите изображение в поиске
2. ПКМ → “Сохранить изображение как...”
3. Создайте папку **my-site/img/** и сохраните в неё
4. Переименуйте файл согласно требованиям выше

Важно: Убедитесь, что папка **img** находится внутри **my-site**, рядом с HTML-файлами!



Пример относительного пути к изображению:

```

```

Пример абсолютного пути (ссылка берётся из интернета):

```

```

Почему важно указывать alt, width и height

- **alt:**

- отображается при ошибке загрузки;
- улучшает доступность и SEO;

- **width и height:**

- резервируют место под изображение;

- предотвращают «прыжки» страницы при загрузке CSS и изображений.

Атрибут **loading="lazy"**.

- изображение загружается только при прокрутке;
- ускоряет загрузку страницы;
- снижает нагрузку на браузер.

Пример с атрибутом **loading**:

```

```

Сделайте скриншот с новыми тегами внутри тела **body** и сохраните файл в папку **img**:

gitLinkLab6_FIO.png

Выполните в терминале:

git add .

git commit -m "Added links and images examples"

git push

Шаг 7. Цитаты в HTML — теги **blockquote**, **cite**. Атрибут **cite**. Источник цитаты в HTML

HTML предоставляет специальные теги для оформления цитат:

➤ **<blockquote>** — используется для **длинных цитат**, оформляется как отдельный блок;

➤ **<cite>** — указывает **источник цитаты** (книга, автор, сайт);

➤ атрибут **cite** — содержит **ссылку на источник**, но визуально не отображается.

Скопируйте из интернета описание ЭВМ. Пример оформления цитаты с источником:

```
<blockquote cite="https://ru.wikipedia.org/wiki/ЭВМ">
  «Электронная вычислительная машина — комплекс технических средств,
  предназначенных для автоматической обработки информации.»
</blockquote>
<cite>Источник: Википедия</cite>
```

Пример цитаты внутри текста

```
<p>
  По словам учёных:
  <q>развитие вычислительной техники изменило ход истории</q>.
</p>
```

Шаг 8. Медиаконтент в HTML — видео, аудио, фреймы. Теги video, audio, iframe. Тег source

HTML позволяет встраивать мультимедиа без сторонних плагинов:

- **<video>** — добавление видео;
- **<audio>** — добавление аудио;
- **<iframe>** — встраивание внешнего контента (YouTube, карты и т.д.);
- **<source>** — указывает источники медиафайлов.

Пример видео:

https://www.w3schools.com/html/mov_bbb.mp4

```
... <video controls width="400">
...   <source src="https://www.w3schools.com/html/mov_bbb.mp4"
type="video/mp4" />
...   Ваш браузер не поддерживает видео.
... </video>
```

Пример аудио:

<https://www.w3schools.com/html/horse.mp3>

```
... <audio controls>
...   <source src="https://www.w3schools.com/html/horse.mp3"
type="audio/mpeg" />
...   Ваш браузер не поддерживает аудио.
... </audio>
```

Пример iframe (YouTube):

<https://www.youtube.com/embed/dQw4w9WgXcQ?si=lWn3NvMZnZ1CcRhp>

```
<iframe
  width="560"
  height="315"
  src="https://www.youtube.com/embed/dQw4w9WgXcQ?si=lWn3NvMZnZ1CcRhp"
  title="История ЭВМ"
  allowfullscreen>
</iframe>
```

Также в качестве примера можно поставить **iframe** из **яндекс.карт**:



Примечание: Для реальных проектов видео и аудио лучше размещать на специализированных сервисах (YouTube, SoundCloud) и встраивать через `<iframe>`.

Шаг 9. HTML тег `input` и его вариации. Атрибут `type`, значения `tel`, `email`, `password`, `url`, `search` и другие

Тег `<input>` используется для ввода данных пользователем.

Атрибут `type` определяет тип вводимой информации.

```
<form>
  <label> Телефон: <input type="tel" placeholder="+7 (___) ___-__-__" /> </label>
  <br /><br />
  <label> Email: <input type="email" placeholder="example@mail.com" /> </label>
  <br /><br />
  <label> Пароль: <input type="password" /> </label>
  <br /><br />
  <label> Сайт: <input type="url" placeholder="https://example.com" /> </label>
  <br /><br />
  <label> Поиск: <input type="search" placeholder="Поиск..." /> </label>
  <br /><br />
  <input type="submit" value="Отправить" />
</form>
```

Типы полей ввода:

Тип	Описание	Пример
<code>text</code>	Обычное текстовое поле	<code><input type="text"></code>
<code>password</code>	Поле для пароля (скрывает символы)	<code><input type="password"></code>
<code>email</code>	Поле для email (проверяет формат)	<code><input type="email"></code>
<code>number</code>	Только цифры	<code><input type="number"></code>
<code>date</code>	Выбор даты	<code><input type="date"></code>
<code>checkbox</code>	Чекбокс	<code><input type="checkbox"></code>
<code>radio</code>	Радиокнопка	<code><input type="radio"></code>
<code>submit</code>	Кнопка отправки формы	<code><input type="submit"></code>

Сделайте скриншоты с новыми тегам внутри тела `body` и сохраните в папку `img`:

gitMediaFormLab6_1_FIO.png

Выполните в терминале:

git add .

git commit -m "Added quotes, media and form inputs"

git push

Шаг 10. Создание простой страницы про историю ЭВМ

Задача: Создать семантически верную HTML-страницу, демонстрирующую использование основных тегов для структурирования контента.

1. Переименуйте существующий файл **index.html** в **practice.html** (чтобы не потерять предыдущую работу).

2. Создайте новый файл с именем **index.html** — это будет наша главная страница.

```
<body>
  <header></header>
  <nav></nav>
  <main>
    <section id="gen1"></section>
    <aside></aside>
    <section id="gen2"></section>
    <aside></aside>
  </main>
  <footer></footer>
</body>
```

Структура: Внутри нового **index.html** необходимо построить структуру веб-страницы, используя предложенные теги. Каждый тег имеет свое логическое назначение:

- **<header>** — для вводного заголовка всей страницы.
- **<nav>** — для навигационного

меню с ссылками на разделы (#gen2) и другие страницы (media.html, forms.html).

- **<main>** — основной контент страницы, включающий информационные разделы (**<section>**) и боковые панели (**<aside>**).

➤ **<section>** — логический раздел с уникальным id.

➤ **<aside>** — боковая панель для дополнительной, связанной информации (в данном случае — изображения поколений ЭВМ).

➤ **<footer>** — подвал страницы с информацией о работе.

Лайфхак: для быстрого добавления через *Emmet* можно использовать следующий вариант:

header^nav^main>section#gen1+aside+section#gen2+aside^footer

Преимущества семантической вёрстки:

1. SEO (поисковая оптимизация)

➤ Поисковые системы лучше понимают структуру страницы

➤ **<header>** и **<h1>** важнее, чем **<div>** с классом "header"

2. Доступность (accessibility)

➤ Программы чтения с экрана (для слабовидящих) правильно озвучивают

контент

- `<nav>` автоматически распознаётся как навигационное меню

3. Читаемость кода

4. Проще поддерживать код

Для вставки используйте следующий текст:

История развития ЭВМ
Поколения
Медиа
Формы
SVG
Первое поколение ЭВМ
Пример якорной ссылки и текстового блока.
ЭВМ первого поколения занимали целые помещения.
Источник: Википедия
Написать письмо Позвонить
Фото
1 поколение ЭВМ
Второе поколение ЭВМ
Пример якорной ссылки и текстового блока.
Во втором поколении ЭВМ появились ОС.
Источник: Википедия
Написать письмо Позвонить
Фото
2 поколение ЭВМ
Лабораторная работа №6 · HTML + CSS

header:

```
8  <body>
9  <header>
10 + <h1>История развития ЭВМ</h1>
11 </header>
```

nav:

```
<nav>
+ <a href="#gen2">Поколения </a>
  <a href="media.html">Медиа</a>
  <a href="forms.html">Формы</a>
  <a href="vector.html" target="_blank">SVG</a>
</nav>
```

Внутри main → section:

```
<main>
+ <section id="gen1">
  <h2>Первое поколение ЭВМ</h2>
  <p>Пример якорной ссылки и текстового блока.</p>
  <blockquote cite="https://ru.wikipedia.org/wiki/ЭВМ">ЭВМ
первого поколения занимали целые помещения.</blockquote>
  <cite>Источник: Википедия</cite>
  <br /><br />
  <a href="mailto:student@mail.ru">Написать письмо</a>
  <a href="tel:+79990000000">Позвонить</a>
  <button>Кнопка</button>
</section>
```

Внутри используются:

- **<blockquote>** — для выделения цитаты с указанием источника в атрибуте `cite`.
- **<cite>** — для подписи источника цитаты.
- Ссылки **mailto:** и **tel:** — для интерактивных действий «написать письмо» и «позвонить».

Далее внутри **main** → **aside**:

```
<main>
  <section id="gen1"> ...
</section>
<aside>
  <h3>Фото</h3>
  
</aside>
```

Далее внутри **main** второй → **section**:

```
<section id="gen2">
  <h2>Второе поколение ЭВМ</h2>
  <p>Пример якорной ссылки и текстового блока.</p>
  <blockquote cite="https://ru.wikipedia.org/wiki/ЭВМ">
    Во втором поколении ЭВМ появились ОС.</blockquote>
  <cite>Источник: Википедия</cite>
  <br /><br />
  <a href="mailto:student@mail.ru">Написать письмо</a>
  <a href="tel:+79990000000">Позвонить</a>
  <button>Кнопка</button>
</section>
```

Затем внутри **main** второй → **aside**:

```
<aside>
  <h3>Фото</h3>
  
</aside>
```

И осталось обновить в конце **footer**:

```
43 </main>
44 <footer>
45   <p title="Подсказка">Лабораторная работа №6 · HTML + CSS</p>
46 </footer>
```

В итоге (проверьте отображение в браузере): получается корректно размеченная, но визуально простая HTML-страница. Все элементы отображаются в стандартном стиле браузера. Именно для стилизации этой структуры и нужен CSS.

Выполните в терминале:

git add .

git commit -m "Added index.html"

git push

Шаг 11. Введение в CSS

Что такое CSS? CSS (Cascading Style Sheets) — это язык стилей, который описывает, как HTML-элементы должны отображаться на экране. Он позволяет управлять цветами, шрифтами, отступами, расположением блоков и многим другим, полностью преобразуя внешний вид страницы.

Что означает “Cascading” (Каскадность)?

CSS применяет стили по приоритету. Если одному элементу задано несколько стилей, применяется самый специфичный.

Теория. Основы синтаксиса CSS

CSS состоит из набора **правил**. Каждое правило определяет, как должны выглядеть определенные элементы на странице.

Подключение стилей

В блок `<head>` файла **index.html** добавьте строку:

```
<head>
  <meta charset="UTF-8" />
  <title>История ЭВМ</title>
  + <link rel="stylesheet" href="style.css" />
</head>
```

Это связывает HTML-документ с внешним файлом стилей **style.css**.

Почему стили в `<head>`?

Тег `<link>` обязательно должен быть в блоке `<head>`, потому что:

1. Браузер загружает стили **ДО** отображения страницы

➤ Если стили в `<body>`, страница сначала отобразится без стилей, потом “прыгнет”

➤ Это называется FOUC (Flash Of Unstyled Content) — плохая практика

2. **<head>** — место для метаданных

➤ Кодировка (`<meta charset="UTF-8">`)

➤ Заголовок (`<title>`)

➤ Стили (`<link>` или `<style>`)

➤ Скрипты (иногда)

Важно: Атрибут href должен содержать правильный путь к файлу CSS.

Структура CSS-правила:

```
селектор {  
  свойство1: значение;  
  свойство2: значение;  
}
```

Основные типы селекторов

Селекторы — самый важный механизм CSS. Они определяют «цель» для стилей.

1. Селектор по тегу (элементу): Выбирает все элементы с данным именем тега.

```
p {  
  font-size: 24px; /* Применится ко ВСЕМ абзацам <p> на странице */  
}
```

2. Селектор по классу (class): Выбирает все элементы, у которых в атрибуте **class** указано нужное имя. В CSS перед именем класса ставится точка .

Добавьте в **html** для любой кнопки класс:

```
<button class="btn-primary">Кнопка</button>
```

Далее в CSS применим стили:

```
.btn-primary {  
  background-color: blue;  
  color: white;  
}
```

Стиль применятся и к кнопке, и к ссылке, так как у обоих **class="btn-primary"**

[Написать письмо](#) [Позвонить](#) Кнопка

3. Селектор по идентификатору (id): Выбирает ОДИН уникальный элемент с указанным id. В CSS перед именем id ставится решетка #.

```
#gen1 {  
  border: 2px solid black;  
}
```

Важное отличие: class можно использовать много раз на странице, id должен быть уникальным.

4. Универсальный селектор: Выбирает **все элементы** на странице.

```
* {  
  /* Часто используется для упрощения расчетов ширины */  
  box-sizing: border-box;  
  /* Например, чтобы убрать все стандартные отступы */  
  margin: 0;  
}
```

Пример приоритетов (от высокого к низкому). Переписывать не нужно

1. Inline-стили (прямо в HTML):

```
<p style="color: red;">Красный текст</p>
```

2. ID-селектор:

```
#main-title {  
  color: blue;  
}
```

3. Class-селектор:

```
.title {  
  color: green;  
}
```

4. Tag-селектор:

```
h1 {  
  color: black;  
}
```

Сделайте скриншот с кодом внутри файла **style.css** и сохраните в папку **img:**
styleCSSLab6_basics_FIO.png

Выполните в терминале:

git add .

git commit -m "Added image for style.css"

git push

Шаг 12. Наглядная демонстрация стилизации

Давайте пошагово проанализируем, как предложенный CSS-код превращает нашу сырую HTML-разметку в стилизованную веб-страницу.

Удалим всё, что написали в файле **style.css**

Перейдём к написанию CSS в файле **style.css**. Для изменений, после каждого введённых правок проверяйте страницу в браузере.

Шаг А: CSS-переменные и сброс отступов

```
*{
  box-sizing: border-box;
}

body{
  margin: 0;
  font-family: Arial, Helvetica, sans-serif;
  background: #f5f7fa;
  color: #333;
}
```

- **box-sizing** — упрощает расчет ширины элементов с padding и border;
- **margin** — убирает стандартные отступы вокруг всей страницы;
- **font-family** — задает шрифт для всего текста;
- **background** — меняет цвет фона страницы;
- **color** — меняет основной цвет текста.

Что такое **box-sizing: border-box**?

По умолчанию браузер считает ширину элемента **без учёта** padding и border:

Без border-box (по умолчанию, **переписывать не нужно**):

```
.box{
  width: 200px;
  padding: 20px;
  border: 5px solid black;
}
```

Реальная ширина: $200 + 20 \times 2 + 5 \times 2 = 250\text{px}$

С border-box: реальная ширина: **200px** (padding и border включены)

Вывод: Применение border-box ко всем элементам (*) упрощает расчёты и делает вёрстку предсказуемой.

Что изменилось: Страница «прижалась» к краям браузера, фон стал светло-серым, а текст — темнее и единообразным.

Шаг В: Стилизация шапки, подвала и навигации

```
header,
footer {
  background-color: #2a6df4;
  color: white;
  padding: 15px;
}

nav {
  background-color: #eef2ff;
  padding: 10px;
  display: flex;
  gap: 10px;
}

nav a {
  text-decoration: none;
  color: #2a6df4;
  font-weight: bold;
}

nav a:hover {
  text-decoration: underline;
}
```

- **padding** — внутренние отступы;
- **display: flex** — включает flex-раскладку для ссылок (CSS Flexbox — это технология для создания сложных гибких макетов за счёт правильного размещения элементов на странице. Подробнее в следующей лабораторной работе).
- **gap** — отступы между ссылками;
- **text-decoration: none** — убирает стандартное подчеркивание ссылок;
- **font-weight: bold** — жирный шрифт;
- **text-decoration: underline** — подчеркивает ссылку при наведении курсора.

Что изменилось: Шапка и подвал стали яркими синими с белым текстом. Навигационная панель превратилась в голубую горизонтальную полосу, где ссылки стали синими кнопками без подчёркивания, но реагируют на наведение.

Шаг С: Сетка для основного содержимого (Grid Layout)

```
main {
  display: grid;
  grid-template-columns: 3fr 1fr;
  gap: 20px;
  padding: 20px;
}

section,
aside {
  background-color: white;
  padding: 15px;
  border-radius: 6px;
  border: 1px solid #d0d7e2;
}
```

- **display: grid** — включает CSS Grid (Грид-раскладка (CSS Grid Layout) представляет собой двумерную систему сеток в CSS);

- **grid-template-columns: 3fr 1fr** — создаёт 2 колонки: широкая (75%) и узкая (25%);
- **gap** — расстояние между колонками и рядами.
- **border-radius** — скругление углов;
- **border** — серая рамка вокруг блоков.

Что такое CSS Grid?

CSS Grid Layout — это мощная система двумерной вёрстки, которая позволяет легко создавать сложные макеты по строкам и столбцам. В отличие от Flexbox (который ориентирован на одномерную раскладку — либо по строке, либо по колонке), Grid работает сразу в двух измерениях. **Подробнее в следующей лабораторной работе.**

Основные понятия

1. Контейнер (Grid Container)

Это родительский элемент, к которому применяется `display: grid`. Он определяет сетку.

2. Элементы сетки (Grid Items)

Это прямые дочерние элементы контейнера. Они автоматически становятся "ячейками" сетки.

Создание сетки

Чтобы задать структуру, используются свойства:

- **grid-template-columns** — определяет столбцы
- **grid-template-rows** — определяет строки

Пример (переписывать не нужно): сетка из 3 колонок по 100px и 2 строк по 50px:

```
.container {  
  display: grid;  
  grid-template-columns: 100px 100px 100px;  
  grid-template-rows: 50px 50px;  
}
```

Можно использовать более гибкие единицы:

- **fr** — доля свободного пространства
- **auto** — размер по содержимому
- **%, px, em** и т.д.

Что такое fr (fractional unit)?

fr — это доля (fraction) от доступного пространства в Grid-контейнере.

Например:

➤ *grid-template-columns: 1fr 1fr;* - делит пространство на 2 равные части (50% + 50%)

➤ *grid-template-columns: 2fr 1fr;*

- Первая колонка: 2 доли (66.67%)

- Вторая колонка: 1 доля (33.33%)

Обязательно подробнее изучите - https://www.w3schools.com/css/css_grid.asp

Что изменилось кардинально: Контент перестал идти одним столбцом. Теперь **<section>** (основной текст) и **<aside>** (фото) расположены **бок о бок** в виде двух колонок. Все блоки получили белую «карточку» с рамкой и скругленными углами, что визуально отделяет их от фона.

Шаг D: Стилизация кнопок и цитат

```
button {
  background-color: #2a6df4;
  color: white;
  border: none;
  padding: 8px 14px;
  cursor: pointer;
  border-radius: 4px;
}

button:hover {
  opacity: 0.85;
}

blockquote {
  border-left: 4px solid #2a6df4;
  padding-left: 10px;
  margin: 10px 0;
  font-style: italic;
}
```

➤ **cursor: pointer** — меняет курсор на «руку» при наведении;

➤ **border-radius: 4px** — скругление кнопок;

➤ **opacity: 0.85** — легкое затемнение кнопки при наведении.

➤ **border-left** — синяя вертикальная полоса слева;

➤ **font-style: italic** — курсив для цитаты.

Что изменилось: Серые квадратные кнопки браузера превратились в стильные синие кнопки с закруглёнными краями и анимацией при наведении. Цитаты теперь четко выделены синей линией слева и курсивом, что соответствует общепринятым стандартам оформления.

Итог: Применение CSS-кода полностью преобразило страницу. Из простого черно-белого документа она превратилась в современный, структурированный и визуально

приятный веб-интерфейс с цветовой схемой, сеточной раскладкой и интерактивными элементами.

Сделайте скриншот с кодом внутри файла **style.css** и сохраните в папку **img**:

styleCSSLab6_layout_FIO.png

Выполните в терминале:

git add .

git commit -m "Added CSS to index.html"

git push

Шаг 13. Редактирование и стилизация страницы форм

Теперь изменим нашу страницу **forms.html**, которая демонстрирует использование основных HTML-элементов для построения интерактивных веб-форм, и применим к ней базовые CSS-стили для улучшения внешнего вида и юзабилити.

В раздел <head> необходимо добавить:

```
<head>
  <meta charset="UTF-8" />
  <title>Формы</title>
  <link rel="stylesheet" href="style.css" />
</head>
```

➤ **Заголовок страницы** который отобразится во вкладке браузера.

➤ **Ссылку на общую таблицу стилей.** Это гарантирует, что стили шапки, подвала, навигации и основного макета будут такими же, как на главной странице.

Разметка тела страницы и формы. Можете воспользоваться вставкой через Emmet:

```
header>h1^^main>section>form>fieldset>legend{Регистрация}
+label*4+label>select>optgroup>option*2^^^label+button^^^aside>p^^footer>a[href]
```

```

<body>
  <header>
    <h1>HTML Формы</h1>
  </header>
  <main>
    <section>
      <form>
        <fieldset>
          <legend>Регистрация</legend>
          <label>
            Имя <input type="text" placeholder="Ваше имя" required />
          </label>
          <label>
            Email <input type="email" autocomplete="email" />
          </label>
          <label>
            Пароль <input type="password" readonly value="12345" />
          </label>
          <label>
            Комментарий <textarea placeholder="Текст"></textarea>
          </label>
          <label>
            Поколение ЭВМ
            <select>
              <optgroup label="ЭВМ">
                <option>Первое</option>
                <option>Второе</option>
              </optgroup>
            </select>
          </label>
          <label>
            Число <input type="number" inputmode="numeric" disabled />
          </label>
        </fieldset>
        <button type="submit">Отправить</button>
      </form>
    </section>
    <aside>
      <p>Пример обязательных и отключённых полей</p>
    </aside>
  </main>
  <footer>
    <a href="index.html">← Назад</a>
  </footer>
</body>

```

Описание структуры: Тело страницы (<body>) сохраняет общую логическую структуру: <header> для заголовка, <main> для основного содержимого, <footer> для

навигации назад. Внутри `<main>` используется `<section>` для самой формы и `<aside>` для пояснительного текста.

Ключевой элемент — `<form>`: Вся группа интерактивных элементов обёрнута в тег `<form>`, который определяет интерактивную форму для сбора пользовательских данных.

Стилизация формы с помощью CSS

Чтобы форма выглядела аккуратно и была удобной для пользователя, необходимо добавить специализированные CSS-правила. Их можно разместить в общем файле `style.css` в конце.

```
form {
  max-width: 400px;
}

label {
  display: block;
  margin-bottom: 10px;
}

input,
textarea,
select {
  width: 100%;
  padding: 6px;
  margin-top: 4px;
}
```

Добавьте следующий код в style.css:

- **max-width: 400px** — ограничивает максимальную ширину формы, не давая ей растягиваться на весь экран;
- **display: block** — превращает метки (`<label>`) в блочные элементы (каждая занимает отдельную строку);
- **margin-bottom: 10px** — создает вертикальный отступ между строками формы;
- **width: 100%** — растягивает поля ввода на всю доступную ширину их контейнера;
- **padding: 6px** — добавляет внутренние отступы в поля для удобства ввода текста;
- **margin-top: 4px** — создает небольшой отступ между текстом метки и самим полем ввода.

Выполните в терминале:

git add .

git commit -m "Change forms.html and CSS"

git push

Шаг 14. Редактирование и стилизация страницы медиа

Теперь изменим нашу страницу `media.html`, которая демонстрирует работу с различными типами медиаконтента в HTML, и применить к ней базовые CSS-стили для улучшения внешнего вида.

В файле `media.html` необходимо убедиться, что в разделе `<head>` присутствуют:

```
<head>
  <meta charset="UTF-8" />
  <title>Медиаконтент</title>
  <link rel="stylesheet" href="style.css" />
</head>
```

➤ **Заголовок страницы** который отобразится во вкладке браузера.

➤ **Ссылку на общую таблицу стилей.** Это гарантирует, что стили шапки, подвала, навигации и основного макета будут такими же, как на главной странице.

Разметка тела страницы

```
<body>
  <header>
    <h1>Медиаконтент HTML</h1>
  </header>
  <main>
    <section>
      <h2>Видео</h2>
      <video controls>
        <source src="video.mp4" type="video/mp4" />
      </video>
    </section>
    <aside>
      <h2>Аудио</h2>
      <audio controls>
        <source src="audio.mp3" type="audio/mpeg" />
      </audio>
      <h2>Iframe</h2>
      <iframe src="https://example.com" height="150"></iframe>
    </aside>
  </main>
  <footer>
    <a href="index.html">← Назад</a>
  </footer>
</body>
```

Описание структуры: Тело страницы (**<body>**) сохраняет общую логическую структуру: **<header>** для заголовка, **<main>** для основного содержимого, **<footer>** для навигации назад. Внутри **<main>** используется **<section>** для видео и **<aside>** для аудио и дополнительного контента.

Стилизация формы с помощью CSS

Чтобы форма выглядела аккуратно и была удобной для пользователя, необходимо добавить специализированные CSS-правила. Их можно разместить в общем файле **style.css** в конце.

Добавьте следующий код в `style.css`:

```
video {
  display: block; /* Убирает лишнее пространство под видео */
  background: #000; /* Черный фон для области видео */
  min-height: 200px; /* Минимальная высота для видео-блока */
}

audio {
  width: 100%; /* Растягивает аудиоплеер на всю ширину */
  margin: 15px 0; /* Отступы сверху и снизу */
}

iframe {
  border: 1px solid #2a6df4; /* Рамка вокруг iframe */
  background: white; /* Фон на случай пустого iframe */
}
```

Выполните в терминале:

git add .

git commit -m "Change media.html and CSS"

git push

Шаг 15. Редактирование и стилизация страницы медиа

Теперь изменим нашу страницу **vector.html**, которая демонстрирует использование масштабируемой векторной графики (SVG) в HTML, и применить к ней базовые CSS-стили для улучшения визуального представления.

В файле **vector.html** необходимо убедиться, что в разделе `<head>` присутствуют:

```
<head>
  <meta charset="UTF-8" />
  <title>SVG</title>
  <link rel="stylesheet" href="style.css" />
</head>
```

➤ **Заголовок страницы** который отобразится во вкладке браузера.

➤ **Ссылку на общую таблицу стилей.** Это гарантирует, что стили шапки, подвала, навигации и основного макета будут такими же, как на главной странице.

Разметка тела страницы

```

<body>
  <header>
    <h1>SVG · Графика</h1>
  </header>
  <main>
    <section>
      <svg width="240" height="120" class="logo"
        title="SVG из Figma" tabindex="0">
        <rect width="240" height="120" fill="#eef2ff" />
        <circle cx="120" cy="60" r="40" fill="#2a6df4" />
        <text x="90" y="65" fill="white">SVG</text>
      </svg>
      <p hidden>Скрытый текст</p>
    </section>
    <aside>
      <p>SVG масштабируется без потери качества</p>
    </aside>
  </main>
  <footer id="footer">
    <a href="index.html">← Назад</a>
  </footer>
</body>

```

1. Что мы использовали в SVG:

- **<svg>** — корневой элемент векторной графики с указанием размеров
- **<rect>** — элемент для рисования прямоугольника (фон)
- **<circle>** — элемент для рисования круга с указанием центра и радиуса
- **<text>** — элемент для добавления текста внутри SVG

2. Атрибуты SVG:

- **width, height** — размеры SVG-холста
- **fill** — цвет заливки элементов
- **cx, cy** — координаты центра круга
- **r** — радиус круга
- **x, y** — координаты текста

3. Специальные атрибуты:

- **hidden** — скрывает элемент (параграф)
- **tabindex="0"** — делает SVG фокусируемым с клавиатуры
- **id="footer"** — уникальный идентификатор для элемента

Стилизация формы с помощью CSS

SVG поддерживает множество дополнительных стилей через CSS:

```
svg {  
  display: block; /* Превращает SVG в блочный элемент */  
  margin: 20px auto; /* Центрирует с отступами сверху/снизу */  
  border: 1px solid #d0d7e2; /* Добавляет тонкую рамку вокруг SVG */  
}  
  
/* Пример дополнительных стилей для SVG */  
svg:hover {  
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1); /* Тень при наведении */  
  transition: box-shadow 0.3s ease; /* Плавный переход */  
}  
  
svg text {  
  font-family: Georgia, serif; /* Шрифт для текста в SVG */  
  font-size: 24px; /* Размер текста */  
  font-weight: bold; /* Жирное начертание */  
}
```

Выполните в терминале:

git add .

git commit -m "Change vector.html and CSS"

git push

Самостоятельное задание. Персональная страница-визитка

Цель задания: Создать простую веб-страницу о себе, используя полученные знания HTML и CSS, и опубликовать её на GitHub.

Требования к HTML:

1. ФИО, группа;
2. Раздел «Обо мне» (3-4 предложения);
3. Добавьте хотя бы один свой навык в список;
4. Контакты (email и телефон, можно не настоящие);
5. Заглушка или реальное фото.

Публикация на GitHub

1. Создайте новый репозиторий на GitHub;
2. Название: my-webpage или about-me-page;
3. Добавьте README.md